

Routability-Driven Macro Placement Engine for Modern FPGAs with Complex Cascade Shape and Region Constraints

Hao Gu, Jian Gu, Keyu Peng, Jianli Chen, *Member, IEEE*,
Jun Yang, *Member, IEEE*, and Ziran Zhu, *Member, IEEE*

Abstract—Field-programmable gate array (FPGA) macro placement holds a crucial role within the FPGA physical design flow since it substantially influences the subsequent stages of cell placement and routing. With the increasing number of macros and the complex cascade shape and region constraints imposed by modern FPGAs, the routability and macro placement have become much more challenging. In this paper, we propose an effective and efficient routability-driven macro placement algorithm for modern FPGAs with cascade shape and region constraints. To reserve adequate space for cell placement and guarantee routability, we first develop a routability-driven mixed-size analytical global placement that evenly distributes both macros and cells while considering cascade shape and region constraints. Particularly, the proposed global placement engine integrates a well-trained congestion prediction model, targeting benchmarks with high routing congestion to enhance overall routability. Then, we propose an integer linear programming (ILP)-based cascade shape legalization followed by matching-based macro legalization to remove macro overlaps while satisfying the region constraints. Finally, a routability-driven detailed macro placement is proposed to refine the solution. Compared with the winners of the MLCAD 2023 FPGA macro placement contest and state-of-the-art works, experimental results show that our algorithm achieves the best overall score and routability.

Index Terms—Electronic design automation, Physical design, FPGA, Placement, Routability, Cascade shape, Legalization.

I. INTRODUCTION

Field-programmable gate arrays (FPGAs) have gained considerable significance in the field of digital design owing to their reconfigurability and versatility. A modern FPGA typically incorporates millions of heterogeneous instances, including fundamental components such as look-up tables (LUTs), flip-flops (FFs), input/outputs (I/Os), digital signal processors (DSPs), and random access memories (RAMs). FPGA macro placement, specifically the placement of DSPs and RAMs, plays a crucial role in the FPGA physical design flow as it significantly influences subsequent stages of cell placement (i.e., the placement of LUTs and FFs) and routing.

Preliminary version of this paper was presented at the 2024 ACM/IEEE Design Automation Conference (DAC'24), San Francisco, CA, USA, June 2024 [6].

This work was partially supported by the National Key Research and Development Program of China under Grant 2022YFB4400500, the National Natural Science Foundation of China under Grant 92373207, and the Zhis-han Young Scholar Support Program of Southeast University under Grant 2240204RCB0052. (*Corresponding author: Ziran Zhu.*)

Hao Gu, Jian Gu, Keyu Peng, Jun Yang, and Ziran Zhu are with the National ASIC System Engineering Research Center, School of Integrated Circuits, Southeast University, Nanjing 210096, China (emails: {230218727, 220216018, kypeng, dragon, zrzh}@seu.edu.cn).

Jianli Chen is with the State Key Lab of ASIC & System, Fudan University, Shanghai 200433, China (email: chenjianli@fudan.edu.cn).

To address the FPGA macro placement, recent advanced works typically employ a mixed-size placement approach that places macros and cells simultaneously. For example, the works [19], [15], [10], [13], [25] first propose mixed-size analytical global placement to find the desired near-legal positions for macros and cells, followed by macro legalization to legalize the RAMs and DSPs. Finally, cell legalization is performed to obtain the final placement results. With more and more macros integrated into modern FPGAs to achieve the performance requirements of advanced applications, the heterogeneous nature of FPGAs poses significant challenges for FPGA placement, especially macro placement.

Besides, routability has emerged as a crucial metric for the placement of modern FPGAs. A placer that does not take routability into account may generate unroutable solutions or lead to severe routing violations, and correcting these violations at the end of the design flow can be challenging and time-consuming. To motivate researchers to tackle the routability issue in FPGA placement, the ISPD 2016 held a routability-driven FPGA placement contest [23]. Many research works [5], [9], [1], [2] have developed advanced algorithms for the contest problem in recent years. However, the problem and benchmark suite released by the contest [23] are based on Xilinx UltraScale architecture, which does not consider the crucial cascade shape and region constraints derived from real-world applications.

Recently, the MLCAD 2023 held an FPGA macro placement contest and provided the benchmarks specifically designed for the Xilinx UltraScale+ architecture [3]. Compared with the ISPD 2016 contest benchmarks, the MLCAD 2023 contest benchmarks not only impose the more practical constraints (i.e., cascade shape and region constraints) based on real-world applications, but also offer an expanded array of DSPs, block RAMs (BRAMs) and ultra RAMs (URAMs) to cater to high-performance signal processing and memory-intensive applications [3]. Due to a sharp increase in the number of macros and the necessity to account for additional cascade shape and region constraints, it is desirable to develop a more practical and efficient macro placer.

Table I illustrates the comparisons of existing placers in different aspects. DREAMPlace-FPGA [19] is designed to handle individual macros and cannot manage cascade shapes that consist of multiple macros and region constraints. Then, DREAMPlaceFPGA-MP [22] consider the MLCAD contest problem and introduced a novel FPGA macro placer that efficiently generates legal macro placement while honoring the specific cascade shapes and region constraints. By treating multiple macros in a cascade shape constraint as a single

TABLE I
COMPARISONS OF EXISTING PLACERS IN DIFFERENT ASPECTS.

Techniques	Global placement			Legalization	
	Cascade handling	Region handling	Instance inflation	Cascade handling	Region handling
DREAMPlace-FPGA [19]	×	×	RUDY-based	×	×
DREAMPlaceFPGA-MP [22]	As single cluster	Force into region	RUDY-based	3-stage bipartite matching	Bipartite matching
OpenPARF 3.0 [14]	As single cluster	Multi-electrostatics region model	RUDY-based	Tetris-based	Bipartite matching
Ours	Pseudo nets & As single cluster	Region function & Force into region	RUDY & ML-based	ILP-based	Bipartite matching

large instance and restricting instances to their respective regions, DREAMPlaceFPGA-MP efficiently handles these complex constraints. However, large clusters may exacerbate the global placement problem, making it more ill-conditioned and difficult to optimize. The enforcement of macros into their assigned regions also can impact the convergence efficiency and quality of the optimization. Besides, OpenPARF 3.0 [14] introduces a robust multi-electrostatics-based FPGA macro placer to achieve better results. OpenPARF 3.0 employs a novel multi-electrostatics region model to handle discontinuities in regions and a divergence-aware density weight scheduling scheme to effectively handle the robustness issues. Experimental results show that it achieves superior placement routability on the public benchmarks of the MLCAD 2023 contest. Furthermore, both DREAMPlaceFPGA-MP and OpenPARF rely solely on RUDY-based congestion estimation and instance inflation, which may limit their ability to optimize routability effectively.

To further improve placement quality and routability, we develop an effective and efficient routability-driven macro placement algorithm for modern FPGAs with cascade shape and region constraints. Our major contributions are summarized as follows:

- To reserve adequate space for cell placement and guarantee routability, we develop a routability-driven mixed-size analytical global placement that evenly distributes both macros and cells while considering cascade shape and region constraints.
- A two-stage cascade shape handling technique is proposed to ensure both the flexibility of instance for spreading and the adherence to cascade shape constraints.
- A region-aware Poisson's equation-based global placement, followed by congestion estimation and instance inflation, is performed alternately to obtain a routing-friendly global placement solution.
- A well-trained congestion prediction model is integrated into our algorithm flow, specifically targeting benchmarks with high routing congestion to further improve their routability.
- To legalize the macro placement and maintain the good solution from the global placement, we propose an ILP-based region-aware cascade shape legalization, followed by matching-based region-aware macro legalization to remove macro overlaps while minimizing the total weighted macro displacement.
- A routability-driven detailed macro placement is presented to refine the routability while preserving the legality of the macro placement.

- Based on the public benchmark suite¹ for the MLCAD 2023 FPGA macro placement contest [3], experimental results show that our algorithm achieves the best overall score and routability among all the contest winners and state-of-the-art works.

The rest of this paper is organized as follows. Section II introduces the target FPGA architecture and constraints, followed by the problem statement. Section III and Section III-A4 details the proposed algorithm. Experimental results are shown in Section IV. Finally, Section V gives the conclusions.

II. PRELIMINARIES

A. Target FPGA Architecture

The target FPGA architecture in this work is the 16nm Xilinx UltraScale+ XCVU3P device, which is also adopted in the MLCAD 2023 FPGA macro placement contest [3]. As shown in Figure 1, a Xilinx UltraScale+ FPGA consists of five site types, i.e., configurable logic blocks (CLBs), IOs, DSPs, BRAMs, and URAMs. Each instance can only be placed in sites of its corresponding type. Each CLB site consists of eight basic logic elements (BLEs) sites, and each BLE site further consists of two LUTs and two FFs sites. Besides, the cascade shape and region constraints imposed by real-world applications must be satisfied [3].

B. Cascade Shape Constraints

To meet the increasing computational and storage demands of advanced applications, cascade shape constraints are introduced by specifying the stacking of DSPs, BRAMs or URAMs [10], [3]. Specifically, each cascade shape constraint requires that specific macros be placed consecutively in a predetermined order. The macros subjected to cascade shape constraints can significantly impact routability and timing closure due to the abundance of pins and net connections [3].

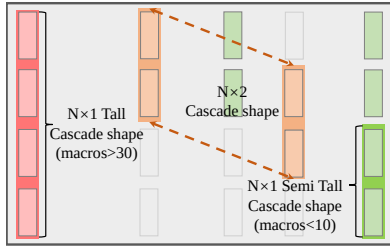
Figure 2(a) illustrates the cascade shape constraints, which can be broadly categorized into three cases: semi-tall, tall, and parallel [21]. For semi-tall and tall cascade shapes ($N \times 1$), macros are arranged according to a predefined order within the same resource type column. A semi-tall cascade shape typically consists of fewer than 10 macros, while a tall cascade shape consists of more than 30 macros [21]. Further, parallel cascade shapes ($N \times 2$) require macros to be positioned in parallel order across two neighbouring columns of the same resource type. Specifically, the parallel cascade shapes can be placed next to each others on separate site column, or on

¹The hidden benchmark suite of the MLCAD 2023 FPGA macro placement contest [3] has not been released.

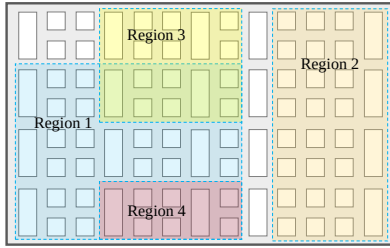


Fig. 1. Xilinx UltraScale+ FPGA architecture.

the same site column. Notably, it is not required for parallel cascade shapes to be placed on the same y-coordinates.



(a) Cascade shape constraints.



(b) Regions constraints.

Fig. 2. Example of cascade shape and region constraints.

C. Region Constraints

To enhance critical design characteristics, some instances with the same function must be placed in a confined region [21], [7]. The region constraints can either be explicitly defined by users or implicitly derived from clock region limitations [21]. Note that the region constraints considered in [7] are “member-hard and non-member-hard”. In other words, instances assigned to a region must be placed inside that region, and instances not assigned to the region are prohibited from appearing in the region. Conversely, for the UltraScale+ architecture considered in this paper, the region constraints consist solely of “member-hard”, signifying that instances assigned to a region must be placed within that region, while instances not assigned to that region can be located anywhere within the placement region [21].

Figure 2(b) illustrates the potential scenarios of regions, which can be broadly categorized into three scenarios: independent regions (e.g., Region 2), overlapping regions (e.g., Region 1 and Region 3), and one region encompassing another (e.g., Region 1 and Region 4).

D. Routability Metric

According to the contest [3], the routability metric consists of two components: (1) initial routing score S_{IR} , and (2) detailed routing score S_{DR} . Note that in order to obtain the S_{IR} and S_{DR} , we need to import the macro placement result into Vivado and adopt Vivado for performing the cell placement and routing.

The S_{IR} is calculated based on the routing congestion level identified by the Vivado initial router. For Xilinx UltraScale+ FPGA device, routing congestion is evaluated within the interconnect tile grid, as shown in Figure 3. Routing congestion occurs when routing resources in some tile grids are overused. Furthermore, routing congestion is classified based on interconnect wirelength (short or long segments) and directions (north, south, east, and west) [3]:

- Short congestion: arising from closely grouped cell placements that potentially pose routability challenges.
- Long congestion: arising from scattered (e.g., poor wirelength) cell placements that pose routability challenges.
- Global congestion: a combined measure of both short and long congestion types.

The short, long, and global congestion levels range from 1 to 8. Level k represents at least one contiguous $2^k \times 2^k$ interconnect tile grid that is overutilized. The S_{IR} is determined by the level in four directions:

$$S_{IR} = 1 + \sum_{d=1}^4 [\max(0, L_{short,d} - 3)^2 + \max(0, L_{global,d} - 3)^2], \quad (1)$$

where $d = 1, 2, 3, 4$ denotes north, south, east and west directions respectively. $L_{short,d}$ and $L_{global,d}$ represent the short and global congestion levels in direction d , respectively. Note that penalties are incurred solely for congestion levels 4 and above.

Furthermore, the S_{DR} is defined as the number of outer iterations performed by the Vivado detailed router. Too many iterations indicate poor macro placement, making it difficult to allocate routing resources. Finally, the overall routability score S_R is defined as:

$$S_R = S_{IR} \times S_{DR}. \quad (2)$$

E. Problem Statement

For the FPGA placement problem, each instance (e.g., FF, LUT, DSP, BRAM, URAM or I/O) can be considered

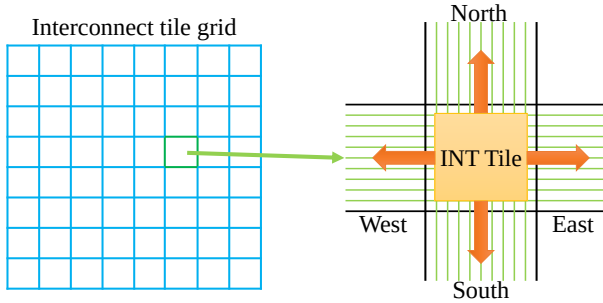


Fig. 3. An example of FPGA interconnect tile grid.

as a vertex; each net can be regarded as a hyperedge, and then a hypergraph $G = (B, E)$ is constructed, where $B = \{b_1, b_2, \dots, b_n\}$ represents the set of n instances, and $E = \{e_1, e_2, \dots, e_m\}$ represents the set of m nets. Let (x_i, y_i) be the coordinate of the center of instance $b_i \in B$. Given the FPGA architecture and a netlist of FFs, LUTs, I/Os, DSPs, BRAMs, URAMs, our goal is to determine the desired position of each macro (e.g., DSP, BRAM, URAM) to optimize the routability and runtime while the cascade shape and region constraints are met.

Specifically, the following constraints must be satisfied:

- 1) The cascade shape constraints must be met.
- 2) The region constraints must be met.
- 3) Instances must be non-overlapping.
- 4) Each instance must be placed in its corresponding resource site.
- 5) The macro placement must be routable by Vivado.

Besides, according to the MLCAD 2023 contest [3], the final evaluation score S_{score} is defined as follows:

$$S_{score} = [1 + \max(0, T_{macro} - 10)] \times S_R \times T_{P\&R}, \quad (3)$$

where T_{macro} denotes the runtime of macro placement in minutes, S_R is the routability score defined in Equation (2), and $T_{P\&R}$ is the runtime of the cell placement and routing performed by Vivado in hours.

III. THE PROPOSED ALGORITHM

Figure 4 provides our algorithm flow for modern FPGAs with cascade shape and region constraints, which mainly includes three parts: (1) routability-driven mixed-size analytical global placement, (2) displacement-driven macro legalization, and (3) routability-driven detailed macro placement. We shall detail these three parts in the following subsections.

A. Routability-Driven Mixed-Size Analytical Global Placement

To reserve adequate space for cell placement and guarantee routability, this subsection details our mixed-size analytical global placement that evenly distributes instances while considering cascade shape and region constraints. Specifically, we first propose a two-stage cascade shape handling technique to tackle the cascade shape constraints. Then, a region-aware

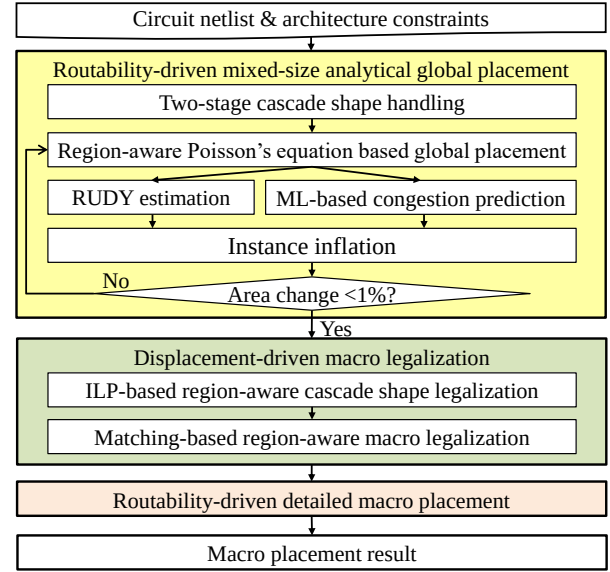


Fig. 4. The overall flow of our proposed algorithm.

Poisson's equation-based global placement, followed by congestion estimation and instance inflation, is performed alternately to obtain a routing-friendly global placement solution.

In the global placement, the overflow ratio is generally used to measure the evenness of instance distribution [19], [25], [15]. The detailed definition of the overflow rate O_t for each resource type t can be found in Equation (7) of [15]. The smaller the O_t , the even the instance distribution. For ease of subsequent presentation, we categorize the overall global placement process into the following two stages based on the instance spreading levels. If $O_t < 0.25$ for $t \in \{DSP, BRAM, URAM\}$ and $O_t < 0.15$ for $t \in \{LUT, FF\}$, we refer to this as the advanced stage of global placement; otherwise, we regard it as the early stage of global placement.

1) *Two-stage cascade shape handling*: To address the cascade shape constraints, an intuitive approach is to merge the macros subjected to each cascade shape constraint into a large cluster, as the approach employed in [10]. However, since there may be dozens of macros within a cascade shape constraint and instances are not substantially spread in the early stage of global placement, the large clusters might obstruct the instance spreading and thus diminish the quality and efficiency of the global placement. To ensure both the flexibility of instance for spreading and the adherence to cascade shape constraints, we propose a two-stage cascade shape handling technique based on different levels of instance spreading.

In the early stage of global placement, since instances are not being sufficiently spread, we introduce pseudo nets with higher net weights for each cascade shape to consider cascade shape constraints while preserving the flexibility of instance spreading. This technique forces the macros within a cascade shape to be placed closer to each other in the predefined order. As illustrated in Figure 5(a) and 5(b), pseudo nets depicted by blue lines are added among macros within the same cascade shape, where Figure 5(a) is semi-tall or tall cascade shape (N

$\times 1$), and Figure 5(b) is parallel cascade shape ($N \times 2$). These pseudo nets contribute to maintaining the predefined order of macros as much as possible during the early stage of global placement, while preventing cascade shapes from obstructing instance spreading. Note that for macros with parallel cascade shape constraints ($N \times 2$), pseudo nets are added between macros within the same column, as shown in Figure 5(b). Further, we dynamically check the relative positions of macros within the same cascade shape constraint at each iteration to avoid potential flips.

In the advanced stage of global placement, since instances are broadly spread, we adopt another cascade shape handling technique. In this stage, macros within the same cascade shape constraint are merged as a single large cluster (outlined by the red dashed line as exhibited in Figures 5(c) and 5(d)). During the merging process, we select the macro with the lowest x-coordinate within the same cascade shape as the bottom-left coordinate of the merged cluster to prevent any flipping. This technique ensures that the cascade shape constraints are met.

Additionally, as shown in Figure 5(d), the relative position of the second column in each parallel cascade shape constraint must be adjacent to or on the same x-coordinate as the position of the first column. Once the $N \times 2$ cascade shape has been merged into two clusters. In each iteration, we use the first cluster c_i (which can be identified by the macro name) as the reference. If the space between these two cluster exceeds the adjacent column of the same resource type, we adjust the position of the second cluster c_j by modifying its x-coordinate x_j to ensure that the adjacent column constraint is satisfied. The adjustment is done as follows:

$$x_j = \begin{cases} x_i^{left}, & \text{if } x_j < x_i^{left} \\ x_j, & \text{if } x_i^{left} \leq x_j \leq x_i^{right} \\ x_i^{right}, & \text{if } x_j > x_i^{right} \end{cases} \quad (4)$$

where x_i^{left} denotes the nearest column coordinate to the left of the first cluster c_i that belongs to the same resource type, and x_i^{right} denotes the nearest column coordinate to the right of x_i that belongs to the same resource type.

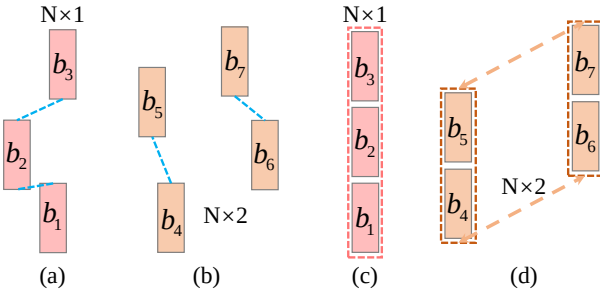


Fig. 5. An example of two-stage cascade shape handling. In (a) and (b), pseudo nets depicted by blue lines are added to handle cascade shape constraints in the early stage of global placement; In (c) and (d), macros subjected to a cascade shape constraint are merged as a cluster in the advanced stage of global placement.

2) *Region-aware Poisson's equation based global placement*: By dividing the placement area into bins, we formulate

the region-aware FPGA global placement as follows:

$$\begin{aligned} \min_{x,y} \quad & WL(x,y) \\ \text{s.t.} \quad & \rho_k(x,y) \leq \rho_m, \quad \forall k \in K, \\ & b_i \subseteq u_i, \quad \forall b_i \in B \end{aligned} \quad (5)$$

where $\rho_k(x,y)$ is the density of bin k , ρ_m is the target density of bin k , u_i is the region constraint of b_i , and $b_i \subseteq u_i$ indicates that block b_i is completely within region u_i . Besides, the wirelength $WL(x,y)$ is defined as the weighted half-perimeter wirelength (HPWL)[4]. Pplace-MS [18] presents a methodologically faster Poisson's equation-based ASIC mixed-size placement engine. Similar to Pplace-MS, the system potential energy in this paper can be calculated by:

$$\mathcal{D}(x,y) = \sum_{b_i \in B} \sum_{t \in T} \iint_{\mathbf{R}(b_i, t)} \psi_t(x,y) dx dy, \quad (6)$$

$\mathbf{R}(b_i, t)$ ($t \in T = \{LUT, FF, DSP, BRAM, URAM\}$) is the intersection region of instance b_i and the region of resource type t , and ψ_t is the electric potential of resource type t . For handling region constraints, we have the following region tension function $F_{u_i}(x_i, y_i)$ of the region u_i for instance b_i :

$$F_{u_i}(x_i, y_i) = f_H(x_i, u_i) + f_V(y_i, u_i), \quad (7)$$

where $f_H(x_i, u_i)$ and $f_V(y_i, u_i)$ are the respective horizontal and vertical region costs. $f_H(x_i, u_i)$ quantifies the horizontal displacement of instance b_i relative to the x-boundaries of region u_i . It is formally expressed as:

$$f_H(x_i, u_i) = \begin{cases} (x_i - x_{u_i}^l)^2, & \text{if } x_i < x_{u_i}^l \\ (x_i - x_{u_i}^h)^2, & \text{if } x_i > x_{u_i}^h \\ 0, & \text{else,} \end{cases} \quad (8)$$

where $(x_{u_i}^l, y_{u_i}^l)$ is the coordinate of lower left corner of u_i , and $(x_{u_i}^h, y_{u_i}^h)$ represents the coordinate of upper right corner. The vertical region cost $f_V(y_i, u_i)$ can be calculated similarly. Based on Equations (6) and (7), the FPGA global placement model (5) can be transformed as follows:

$$\min_{x,y} g(x,y) = W(x,y) + \lambda_d \mathcal{D}(x,y) + \lambda_r F(x,y), \quad (9)$$

where λ_d and λ_r are regularization factors. And the region tension function $F(x,y)$ of system is calculated by:

$$F(x,y) = \sum_{b_i \in B_R} F_{u_i}(x_i, y_i), \quad (10)$$

where B_R denotes the set of instances with region constraints. In addition, a weighted average (WA) model $W(x,y)$ [8] is used to smooth the wirelength function.

For Equation (9), the density regularization factor λ_d is updated by Equations (35) and (36) in [12]. Furthermore, λ_r is determined by:

$$\lambda_r^{j+1} = \lambda_r^j + \Delta m^j, \quad (11)$$

where j denotes the global placement iteration number. Δm^j determines the increment value of λ_r in the next iteration, which is estimated by:

$$\Delta m^j = \begin{cases} \alpha, & j = 0 \\ \frac{\beta}{\|\mathcal{D}(x,y)\|_1}, & j > 0, \end{cases} \quad (12)$$

where α and β are user-specified parameters. Then, based on $\nabla g(x, y)$, a preconditioned Nesterov's method in ePlace [12] is used to optimize the positions of movable instances.

Although the region force $\nabla F(x, y)$ effectively guides instances toward their corresponding regions during global placement, there may still be several instances that fail to satisfy the region constraints. Therefore, during the advanced stage of global placement where instances have been broadly spread, we further impose a forced region shift on the instances with region constraints to ensure that all the region constraints are met. Specifically, for each instance b_i that needs to be placed inside region u_i , we update the position according to the following equation during each iteration of the advanced stage of global placement:

$$\begin{cases} x_i = \min[\max(x_i, x_{u_i}^l), x_{u_i}^h - w_i] \\ y_i = \min[\max(y_i, y_{u_i}^l), y_{u_i}^h - h_i], \end{cases} \quad (13)$$

where w_i and h_i denote the width and height of instance b_i . Besides, note that the two-stage cascade shape handling presented in Section III-A1 is integrated into the process of solving the Equation (9). That is, in the early stage of global placement, pseudo nets with higher net weights are added to consider the cascade shape constraints. Further, in the advanced stage of global placement, macros subjected to a cascade shape constraint are merged as a cluster to ensure the cascade shape constraints are met.

3) *RUDY-based congestion estimation and instance inflation*: To achieve a routing-friendly placement solution, we perform a congestion estimation and instance inflation to optimize the routability during the advanced stage of global placement. Similar to [15], we adopt the rectangular uniform wire density (RUDY) [20] to estimate routing congestion, and then calculate the routability-optimized area for each instance/cluster. Let H_i and V_i represent the horizontal and vertical routing utilization at instance/cluster b_i estimated by RUDY, respectively. For each cluster subjected to the cascade shape constraint, we calculate H_i and V_i by averaging the utilization across multiple routing grids occupied by the cluster. Then, the routability-optimized area for b_i is determined as follows:

$$A_i^{est} = A_i \times \min[\max(H_i, V_i)^{2.5}, \epsilon], \quad (14)$$

where A_i is the current area of b_i , A_i^{est} is the estimated area of b_i , and ϵ is an empirical constant to avoid instances overinflating, which is set to 2 in our implementation. Besides, the inflation factor ΔA_i of b_i is defined as $\Delta A_i = A_i^{est} - A_i$. To prevent the total inflation area from exceeding the total area of each resource type $t \in T$, the inflation area needs to be scaled by the following factors according to its resource type t :

$$\tau_t = \min\left(\frac{A_t^p - \sum_{i \in t} A_i}{\sum_{i \in t} \Delta A_i}, 1\right), \quad (15)$$

where A_t^p is the whole available placement area of resource type t . Then, the final area A_i^{update} of b_i after inflation can be calculated as follows:

$$A_i^{update} = A_i + \tau_t \times \Delta A_i. \quad (16)$$

4) *ML-based congestion prediction for routability optimization*: In this subsection, we present a machine learning (ML)-based congestion prediction model designed to specifically address benchmarks exhibiting high routing congestion, with the goal of further improving their routability. By analyzing the results from our previous conference version DAC'24 [6], we observed a significant improvement in routability for most of the 140 public benchmarks, compared to winners of the contest and other state-of-the-art works. However, Figure 6 shows deviations from the average of metric S_R and S_{score} of all benchmarks, which reveals that 10 benchmarks exhibited notably high routing congestion.

To address high routing congestion, we employ the ML-based congestion prediction for these 10 benchmarks, while the remaining 130 benchmarks are evaluated using RUDY-based congestion estimation. The ML-based model requires significant computational resources for model training, and the runtime of congestion prediction is also longer than that of RUDY. Although multiple congestion predictions increase the total placement time, this addition remains minor compared to the subsequent routing phase.

Additionally, the effectiveness of the ML-based model is closely tied to the training data and generalization capability. When applied to all benchmarks, the prediction accuracy of the ML-based model may not always outperform RUDY due to its limited generalization capability. The placement routability on benchmarks with low routing congestion remains comparable to RUDY, while the ML-based model incurs higher computational time. As a result, by applying an ML-based model to benchmarks with high routing congestion, we can effectively balance performance and efficiency.

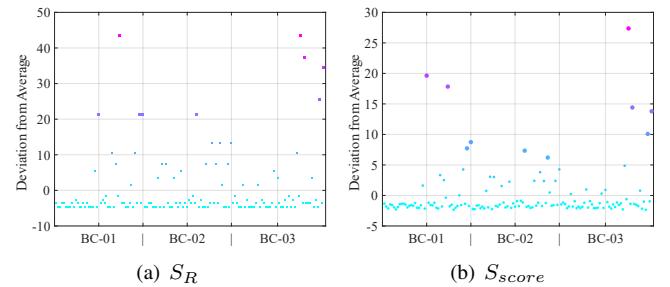


Fig. 6. Deviation from average of S_R and S_{score} in BC-01, BC-02 and BC-03.

We propose an ML-based congestion prediction model by utilizing grid-based placement features and a novel prediction model to predict congestion level maps. The accuracy of congestion prediction is significantly influenced by the input features extracted from the grid-based placement. Based on the placement grids, we identify four critical grid-based features strongly correlated with the congestion level map. Detailed descriptions of each feature are provided below. **Macro Map** indicates the occupancy status of each grid by a macro, which can help identify congestion areas with macro locations. **RUDY Map** is a popular routing demand estimation technique, which is the superposition of horizontal net density and vertical net density. **Pin RUDY Map** refers to a pin density map inspired by RUDY, representing the horizontal

and vertical pin density of all nets within a grid. **Cell Density Map** is defined as the number of cells in each grid.

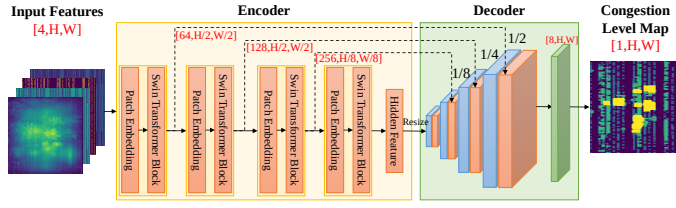


Fig. 7. Framework of our proposed congestion prediction model.

Figure 7 illustrates the framework of our congestion prediction model, which mainly consists of three components: grid-based input features, encoder with transformer layers, and cascaded decoder. Specifically, the model inputs $X_{input} \in \mathbb{R}^{4 \times H \times W}$ consist of grid-based placement features that capture various aspects of circuit placement, with W and H representing width and height of placement grid, respectively. The encoder extracts multiscale features with multiple swin transformer layers [11], which employ dynamic attention mechanisms to extract global information. This feature extraction improves the difference between various congestion levels and enhances the ability to identify them. In the decoder, by integrating the output of the upsampling layer with the multiscale skip connection layer, our model substantially enhances its capability to learn features and recover more precise congestion level maps. Finally, our model can accurately predict the congestion level map $Y_{out} \in \mathbb{R}_+^{1 \times H \times W}$.

More importantly, to further enhance the routability of the global placement results, we utilize our trained congestion prediction model to predict congestion map $Y_{out} \in \mathbb{R}_+^{1 \times H \times W}$ instead of the original RUDY method used in Section III-A3. Given that penalties to S_{IR} , defined in Equation (1), are only incurred when the congestion level is greater than 3, we only inflate the instances in the grid where Y_{out}^i exceeds 3. Hence, the routability-optimized area for each instance/cluster b_i defined in Eq. (14) is modified as follows:

$$A_i^{est} = A_i \times \min[\max(Y_{out}^i - 2, 1)^{2.5}, \epsilon], \quad (17)$$

5) *Model Training*: According to the MLCAD 2023 contest [3], interconnect congestion levels are the most important metric for evaluating routability. Unlike traditional continuous congestion map, FPGA congestion level map consists of discrete values representing interconnect congestion levels. Therefore, we adopt the interconnect congestion level map as the model training label Y_{lab} . Our primary goal is to perform multiclass classification, where the congestion is categorized into eight distinct levels with maximal accuracy. To achieve this, we employ categorical cross-entropy as the loss function, which is widely recognized for its effectiveness in various multiclass classification situations. The categorical cross-entropy loss is mathematically defined as:

$$\mathcal{L}(Y_{out}, Y_{lab}) = -\frac{1}{H \times W} \sum_{i=1}^{H \times W} \sum_{c=1}^M \mathbb{E}_c(Y_{out}^i, Y_{lab}^i) \log(p_c^i), \quad (18)$$

where $H \times W$ and M are the number of grids and congestion levels, respectively. $\mathbb{E}(\cdot)$ means is the equal function that is 1 if the predicted congestion level Y_{out}^i matches true congestion level Y_{lab}^i , and 0 otherwise. p_c^i is the predicted probability that Y_{out}^i belongs to level c .

6) *ML-based Model Evaluation Metrics*: To quantitatively compare the effectiveness of the ML-based model, we evaluate it using three commonly used metrics [24]: structural similarity index measure (SSIM), coefficient of determination (R^2) and normalized root mean square error (NRMS). These metrics provide a comprehensive assessment of the similarity, correlation, and pixel-level accuracy of the predicted congestion maps.

Firstly, SSIM measures the structural similarity between the predicted congestion map Y_{out} and the label Y_{lab} . A higher SSIM value indicates that the predicted congestion distribution closely aligns with the label. Given μ as the mean, σ as the standard deviation, and C_1 and C_2 as stabilizing constants. SSIM is defined as:

$$\text{SSIM}(Y_{out}, Y_{lab}) = \frac{(2\mu_{Y_{out}}\mu_{Y_{lab}} + C_1)(2\sigma_{Y_{out}, Y_{lab}} + C_2)}{(\mu_{Y_{out}}^2 + \mu_{Y_{lab}}^2 + C_1)(\sigma_{Y_{out}}^2 + \sigma_{Y_{lab}}^2 + C_2)}. \quad (19)$$

Secondly, R^2 quantifies how well the predicted congestion values correlate with the label values. A R^2 value closer to 1 indicates a strong correlation between Y_{out} and Y_{lab} . This metric is computed as:

$$R^2(Y_{out}, Y_{lab}) = 1 - \frac{\sum_{m=1}^H \sum_{n=1}^W (Y_{out}^{m,n} - Y_{lab}^{m,n})^2}{\sum_{m=1}^H \sum_{n=1}^W (Y_{lab}^{m,n} - Y_{lab}^{\text{mean}})^2}, \quad (20)$$

where Y_{lab}^{mean} represents the mean value of Y_{lab} .

Finally, NRMS evaluates the pixel-level error between Y_{out} and Y_{lab} . A lower NRMS value indicates higher accuracy in congestion prediction. NRMS is defined as:

$$\text{NRMS}(Y_{out}, Y_{lab}) = \frac{\|Y_{out} - Y_{lab}\|_2}{[\max(Y_{lab}) - \min(Y_{lab})] \sqrt{N_{Y_{lab}}}}. \quad (21)$$

B. Displacement-Driven Macro Legalization

After routability-driven mixed-size analytical global placement, we propose a displacement-driven macro legalization algorithm to legalize all the macros while preserving the good solution provided by global placement as much as possible. Since the clusters (i.e., the macros subjected to cascade shape constraints) have larger sizes and higher connectivity than other macros, it is preferable to legalize cluster macros before other macros. Therefore, our macro legalization consists of two stages, which shall be elaborated on in the subsequent subsection.

1) *ILP-based region-aware cascades shape legalization*: Previous works [15], [19], [25], [5] typically adopted bipartite matching-based approaches to legalize macros. However, these works did not consider the complex cascade shape constraints. As the macros within each cascade shape need to occupy multiple consecutive sites in a predefined order, the matching-based approaches cannot address the cascade shape constraints. Furthermore, since the region constraints in the target FPGA architecture are only “member-hard” and regions exhibit various scenarios (i.e., independent, overlapping, and

TABLE II
NOTATIONS USED IN OUR CASCADE SHAPE LEGALIZATION.

C	The set of clusters.
S	The set of sites.
S_c	The set of alternative groups for cluster c .
N_c	The number of macros in cluster c .
g	A group $g \in S_c$ that contains a set of sites for accommodating c .
P_c	The preconditioner of cluster c .
d_{cg}	The displacement of moving cluster c to group g .
m_{cg}	The binary variable indicates whether group g is selected by cluster c .
n_{cs}	The binary variable indicates whether site s is occupied by cluster c .

encompassing), as illustrated in Section II-C, addressing each region individually may limit the solution quality. To legalize the cascade shapes and handle multiple regions simultaneously, we formulate the region-aware cascade shape legalization as an ILP model in (22). The notations of model (22) are given in Table II. Note that the macros subjected to a cascade shape constraint are regarded as a cluster.

$$\min \quad \sum_{c \in C} \sum_{g \in S_c} m_{cg} \cdot P_c \cdot d_{cg} \quad (22)$$

$$\text{s.t.} \quad \sum_{g \in S_c} m_{cg} = 1, \quad \forall c \in C; \quad (22a)$$

$$\sum_{c \in C} n_{cs} \leq 1, \quad \forall s \in S; \quad (22b)$$

$$\sum_{s \in g} n_{cs} = N_c \cdot m_{cg}, \quad \forall c \in C, \forall g \in S_c; \quad (22c)$$

$$m_{cg} \in \{0, 1\}, \quad \forall c \in C, \forall g \in S_c; \quad (22d)$$

$$n_{cs} \in \{0, 1\}, \quad \forall c \in C, \forall s \in S, \quad (22e)$$

where the preconditioner P_c is defined as follows [15]:

$$P_c = \sum_{e \in E_c} \frac{1}{|e| - 1}, \quad (23)$$

where E_c is the set of nets that connected to cluster c , and $|e|$ is the number of pin in net e . In the ILP model (22), the objective is to minimize the total weighted cluster displacement. Constraint (22a) denotes each cluster c can only choose an alternative group of sites. Constraint (22b) represents each site s can only be occupied at most once. Constraint (22c) denotes that each cluster c occupies N_c sites since the cluster contains N_c macros. Besides, the groups S_c are selected according to the positions after global placement, with only the ten nearest legal groups (e.g., the region constraints are met) qualifying for selection. Given that modern FPGAs typically contain only dozens of cascade shapes, the ILP model can be solved by Gurobi [17] in a reasonable runtime. Note that since the DSP, BRAM, and URAM have respective corresponding sites, they can be solved separately. That is, the ILP model is subsequently solved for DSP cascade shapes, BRAM cascade shapes, and URAM cascade shapes individually.

2) *Matching-based region-aware macro legalization*: After the cascade shapes legalization, the sites occupied by cascade shapes are set as unavailable for other macros, and all macros in cascade shapes are fixed. Then, in order to legalize the macros without cascade shape constraints, we model the macro

legalization problem as follows:

$$\min \quad \sum_{b_i \in B_m} \sum_{s \in S_{b_i}} m_{b_i,s} \cdot P_{b_i} \cdot d_{b_i,s} \quad (24)$$

$$\text{s.t.} \quad \sum_{s \in S_{b_i}} m_{b_i,s} = 1, \quad \forall b_i \in B_m; \quad (24a)$$

$$\sum_{b_i \in B_m} m_{b_i,s} \leq 1, \quad \forall s \in S_{b_i}; \quad (24b)$$

$$m_{b_i,s} \in \{0, 1\}, \quad \forall b_i \in B_m, \forall s \in S_{b_i}. \quad (24c)$$

The notations of Problem (24) are similar to that of Problem (22), where B_m is the set of macros without cascade shape constraints, S_{b_i} denotes all available sites for macro b_i , $m_{b_i,s}$ is a binary variable that indicates whether site s is selected by b_i , P_{b_i} represents the preconditioner of b_i , and $d_{b_i,s}$ denotes the displacement of moving the macro b_i to the site s . Constraint (24a) indicates each macro b_i can only choose one site for placement. Constraint (24b) denotes that each site s can be occupied by at most one macro.

Cascade shapes have different dimensions, while macros of one type have the same size. Because of this property, the legalization of remaining macros can be solved as a min-cost bipartite matching problem. Notably, macros with region constraints can exclusively connect with sites located in their corresponding regions. Then, a bipartite graph $\{B_m, S_{b_i}\}$ is constructed and the edge cost between a macro $b_i \in B_m$ and a site $s \in S_{b_i}$ is determined by:

$$e(b_i, s) = P_{b_i} \cdot d_{b_i,s}. \quad (25)$$

We employ the same method used in [9] to solve this min-cost flow problem. Similar to the cascade shape legalization, the bipartite matching-based legalization is executed three times. Specifically, it is performed separately for DSPs, BRAMs, and URAMs.

C. Routability-Driven Detailed Macro Placement

Although the macro legalization stage strives to maintain the routing-friendly global placement solution by minimizing the total weighted macro displacement, it does not directly consider the routability. Therefore, we present a detailed macro placement in this section to refine the routability directly.

Since the complexity and excessive perturbation may be caused by moving the macros with cascade shape constraints, we enforce the macros with cascade shape constraints are fixed in this stage. The same as the congestion estimation presented in Section III-A3, based on RUDY [20], we first calculate the horizontal and vertical routing utilization H_i and V_i for each macro b_i . Then, we iteratively identify the macro b_i with the highest $\max(H_i, V_i)$ to reallocation. In an attempt to reallocate b_i , the optimal region R_{opt} for macro b_i is determined using the method described in [16]. If available sites exist in R_{opt} , we try to move macro b_i to an available site in R_{opt} . If R_{opt} lacks available sites, we try to swap macro b_i with the macro on an occupied site. Note that macro reallocation occurs only if it can reduce the maximum routing utilization while maintaining the legality of the macro placement. Once a macro is reallocated, it is set as fixed. When three attempts to reallocate different macros fail, this iterative macro reallocation process terminates.

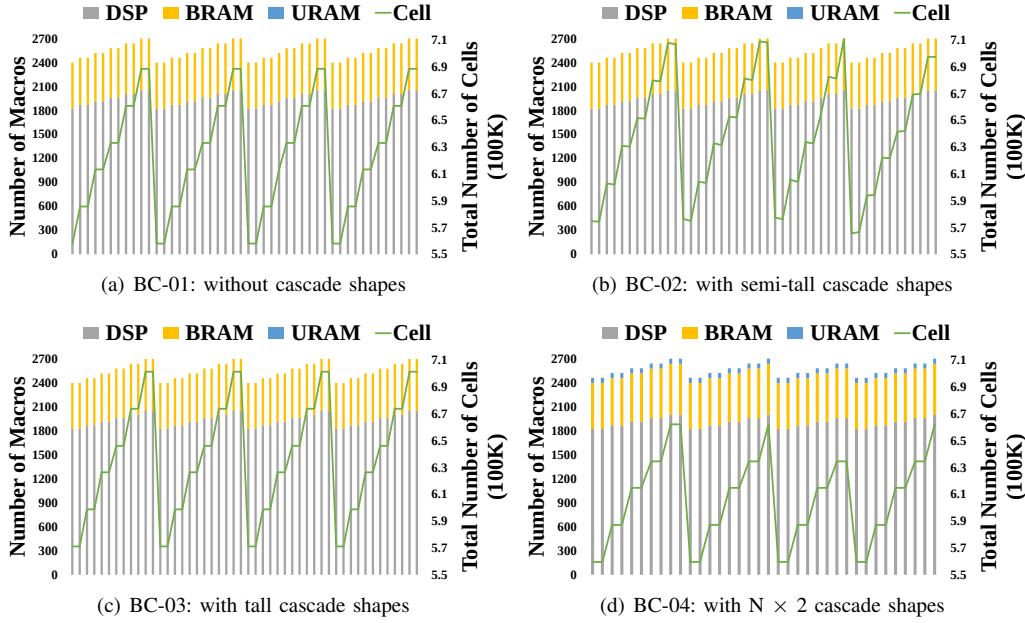


Fig. 8. Overview of public benchmarks in MLCAD 2023 contest.

IV. EXPERIMENTAL RESULTS

A. Benchmarks and Settings

1) *Implementation Details:* We implemented the proposed macro placement algorithm in a combination of *C++* and *Python*, and performed all experiments on a *Ubuntu 20.04* Linux machine with *AMD EPYC 7763 3.5GHz* CPUs and *NVIDIA Geforce 4090* GPUs. Vivado 2021.1 [21] was used to verify the legality of macro placement and perform cell placement and routing. The interconnect congestion level map labels are obtained with the *Export to Spreadsheet* command in Vivado, which serves as the ground truth labels.

2) *Benchmarks:* The benchmark suite for the MLCAD 2023 FPGA macro placement contest [3] includes 176 public benchmarks (140 used for evaluation and 36 excluded from evaluation), along with an additional 198 hidden benchmarks that remain unreleased. The number of macros included in these public benchmarks ranges from 2400 to 2700, and the number of cells included ranges from 550k to 710k. As depicted in Figure 8, the grey, orange, and blue bars represent the number of DSPs, BRAMs, and URAMs, respectively. Overlaying these, the green line shows the total number of cells in each benchmark, scaled on the right *y*-axis in 100K. Notably, the region constraints of these benchmarks are derived from 1, 24, 30, or 38 clock fence regions.

In accordance with cascade shape constraints, we categorize the benchmarks into four groups: “without cascade shape”, “with semi-tall cascade shapes”, “with tall cascade shapes”, and “with $N \times 2$ cascade shapes”, denoted as BC-01, BC-02, BC-03, and BC-04, comprising 46, 46, 48, and 36 benchmarks, respectively. Besides, BC-01, BC-02, and BC-03 (with a total of 140 benchmarks) have been used in the final evaluation of the contest. To further evaluate the performance of the proposed algorithm in $N \times 2$ cascade shapes scenarios, we

also provide the experimental results for BC-04 in Section IV-D.

B. Result Analysis on BC-01, BC-02, and BC-03

In this section, we compare the proposed algorithm with our previous conference version DAC-2024 [6], OpenPARF 3.0 [14], the two first-place teams (i.e., DREAMPlaceFPGA-MP [22] and SEU) in the MLCAD 2023 FPGA macro placement contest [3], and Vivado 2021.1 [21]. With the binary codes provided by the authors of these macro placers, we were able to run all the macro placers on the same machine for a fair comparison². For the 140 benchmarks (including BC-01, BC-02, and BC-03) used for evaluation in the contest, Table III gives the arithmetic mean (denoted by Avg.) and geometric mean (denoted by Geo.) values of the metrics T_{macro} , $T_{P\&R}$, S_{IR} , S_{DR} , S_R , and S_{score} . Compared to arithmetic mean values, which are more sensitive to extreme values, geometric mean values provide a more balanced measure that reflects the typical performance by mitigating the effects of extreme values.

As shown in Table III, although the runtime of our macro placer T_{macro} is slower than DAC-2024, SEU and DREAMPlaceFPGA-MP, it remains below 10 minutes, thus preventing the macro placement runtime penalty in S_{score} . Furthermore, our algorithm has achieved superior results in other metrics, including the runtime of Vivado cell placement

²Please note that the results given in the contest [3] are inconsistent with our own calculations of S_R and S_{score} in Equation (3), possibly due to the differences in machines or the absence of exact evaluation details provided by the contest organizers. Our discussions with the other two top teams (i.e., DREAMPlaceFPGA-MP and MPKU) revealed that they also found performance gaps between the results they obtained and those announced in the contest. As the contest server is currently inaccessible and the contest organizers have not responded to these issues, we conducted all the experiments on our machine and reported the relevant results calculated using Equations (1), (2), and (3).

TABLE III
EXPERIMENTAL RESULTS ON BC-01, BC-02, AND BC-03.

Metrics ¹	T_{macro} (min)		$T_{P\&R}$ (hr)		S_{IR}		S_{DR}		S_R		S_{score}	
	Avg.	Geo.	Avg.	Geo.	Avg.	Geo.	Avg.	Geo.	Avg.	Geo.	Avg.	Geo.
Ours	4.65	4.42	0.35	0.34	1.43	1.26	6.80	6.74	9.94	8.49	3.59	2.91
DAC-2024 [6]	4.63 -0.4% ²	4.42 -0.1%	0.35 +0.4%	0.34 +0.2%	1.49 +4.0%	1.28 +1.4%	6.84 +0.5%	6.77 +0.4%	10.57 +6.3%	8.65 +1.8%	3.91 +8.9%	2.97 +2.1%
OpenPARF 3.0 [14]	8.46 +82.1%	8.45 +91.1%	0.36 +3.6%	0.35 +2.5%	1.57 +10.0%	1.28 +1.3%	7.34 +8.0%	7.22 +7.0%	12.86 +29.3%	9.16 +7.9%	5.40 +50.3%	3.22 +10.6%
SEU³	2.56 -44.9%	2.53 -42.8%	0.38 +10.8%	0.37 +7.9%	1.56 +9.5%	1.30 +3.1%	7.09 +4.3%	6.99 +3.8%	11.88 +19.5%	9.09 +7.0%	5.30 +47.6%	3.36 +15.5%
DREAMPlaceFPGA-MP³ [22]	2.70 -42.6%	2.65 -40.0%	0.37 +7.3%	0.36 +4.7%	1.83 +28.0%	1.36 +8.0%	7.23 +6.3%	7.11 +5.5%	15.27 +53.6%	9.67 +13.9%	7.76 +116.1%	3.47 +19.2%
Vivado [21]	-	-	0.52 +49.4%	0.49 +43.5%	2.30 +61.0%	1.51 +19.9%	8.26 +21.4%	7.99 +18.5%	22.94 +130.7%	12.06 +42.1%	18.69 +420.6%	5.94 +103.8%

¹ All metrics: Lower is better.

² Percentages indicate how much worse (+) or better (-). All placers are compared to ours.

³ DREAMPlaceFPGA-MP (UTDA) and SEU are tied for first place in the contest according to official announcement [3].

and routing $T_{P\&R}$, initial routing score S_{IR} , detailed routing score S_{DR} , overall routability score S_R , and final score S_{score} . Specifically, regarding the overall routability score S_R , our algorithm outperforms DAC-2024, OpenPARF 3.0 and Vivado by 6.3%, 29.3%, and 130.7% in the arithmetic mean, and by 1.8%, 7.9%, and 42.1% in the geometric mean. In terms of the final score S_{score} , our algorithm outperforms DAC-2024, OpenPARF 3.0, SEU, DREAMPlaceFPGA-MP and Vivado by 8.9%, 50.3%, 47.6%, 116.1% and 420.6% in the arithmetic mean, and by 2.1%, 10.6%, 15.5%, 19.2% and 103.8% in the geometric mean.

Besides, Figure 9 provides the detailed comparison of S_R and S_{score} for each benchmark in BC-01, BC-02 and BC-03. As can be seen from the figure, our algorithm achieves the best routability and final score in most benchmarks, and the smaller extreme values also verify the robustness of our algorithm. Overall, the experimental results have shown that our algorithm achieves the best routability and overall score among the contest winners and DAC-2024.

C. Result Analysis of the ML-based model

The evaluation of our ML-based model includes both congestion prediction accuracy and routability-driven macro placement performance.

To evaluate congestion prediction accuracy, Table IV provides a quantitative comparison of RUDY-based and ML-based congestion prediction across 10 benchmarks with high routing congestion. Notably, the training and validation datasets are independently separated to ensure an unbiased evaluation of the performance our model. The results demonstrate that the ML-based congestion prediction model significantly outperforms RUDY in terms of SSIM, R^2 and NRMS. On average, our ML-based model improves SSIM by 59.3%, R^2 by 39.8%, and improves NRMS by 57.4% compared to RUDY. These improvements confirm that the ML-based model is particularly effective in congestion estimation.

Additionally, we evaluate the impact of the ML-based model on the overall routability-driven macro placement per-

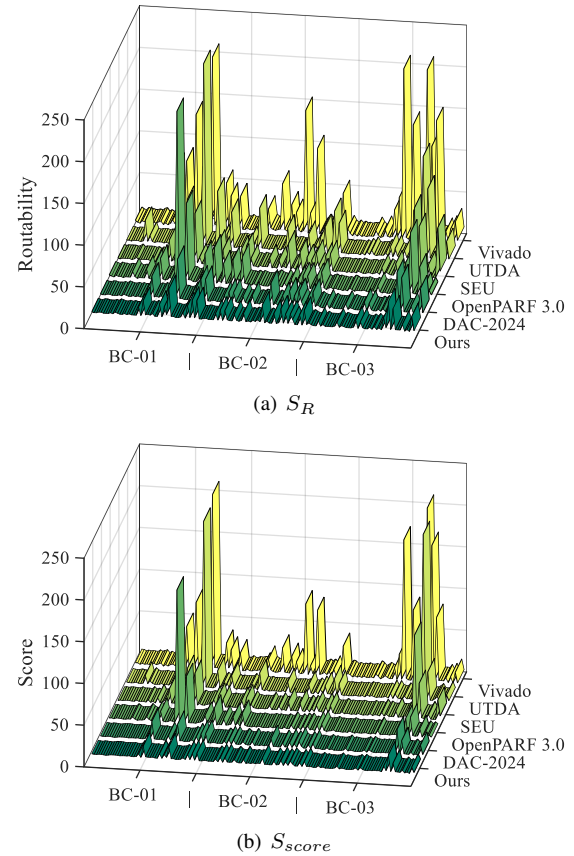
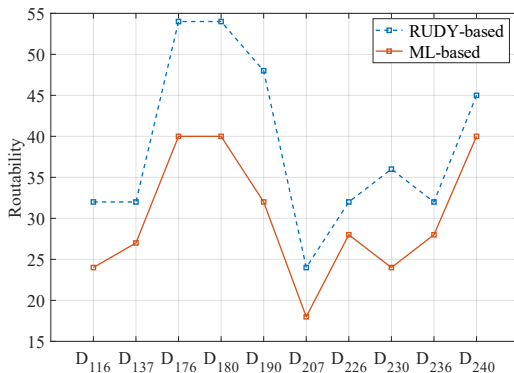


Fig. 9. Comparison of S_R and S_{score} in BC-01, BC-02 and BC-03.

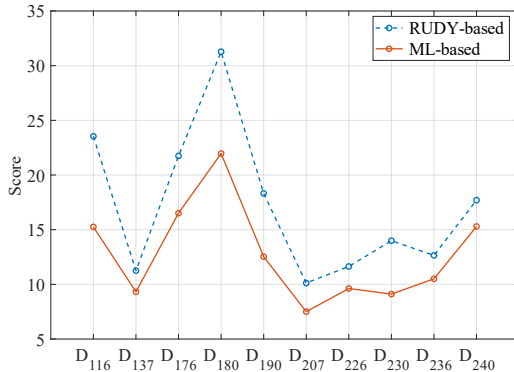
formance. Figure 10 compares the performance of S_R and S_{score} between RUDY-based and ML-based congestion prediction across 10 benchmarks with high routing congestion. These results demonstrate significant improvements in targeted congestion optimization, highlighting the effectiveness of integrating the trained congestion prediction model into our algorithm flow.

TABLE IV
PREDICTION ACCURACY COMPARISON OF RUDY-BASED AND ML-BASED CONGESTION ESTIMATION.

Benchmarks	RUDY-based			ML-based (Ours)		
	SSIM $\uparrow$	R $^2\uparrow$	NRMS $\downarrow$	SSIM $\uparrow$	R $^2\uparrow$	NRMS $\downarrow$
Design_116	0.27	0.50	0.31	0.70	0.82	0.19
Design_137	0.28	0.47	0.28	0.68	0.84	0.19
Design_176	0.23	0.56	0.31	0.63	0.81	0.20
Design_180	0.34	0.56	0.26	0.66	0.88	0.15
Design_190	0.50	0.72	0.29	0.63	0.84	0.13
Design_207	0.34	0.62	0.26	0.71	0.90	0.15
Design_226	0.18	0.35	0.23	0.61	0.74	0.20
Design_230	0.16	0.34	0.35	0.64	0.70	0.20
Design_236	0.25	0.52	0.32	0.69	0.79	0.20
Design_240	0.14	0.25	0.35	0.67	0.79	0.26
Average Ratio	0.270	0.488	0.295	0.662	0.811	0.188
	0.407	0.602	1.574	1.000	1.000	1.000



(a) S_R



(b) S_{score}

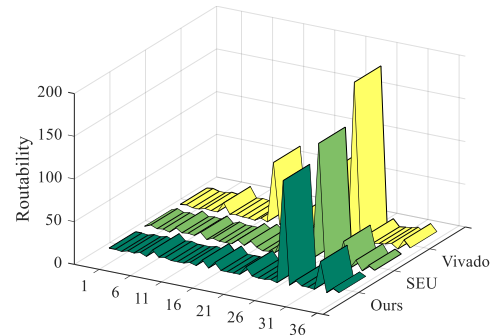
Fig. 10. Performance comparison of ML-based model in S_R and S_{score} .

D. Result Analysis on BC-04

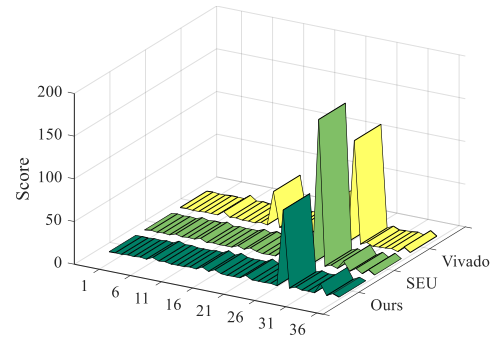
Although BC-04 was not included in the final evaluation of the contest [3], the experimental results for BC-04 are also given in Table V to further evaluate the performance and robustness of our algorithm in $N \times 2$ cascade shapes scenarios. Since the binary codes provided by other macro placers cannot tackle the $N \times 2$ cascade shape constraints, we only compare with SEU and Vivado in Table V.

As shown in Table V, although SEU achieves the shorter T_{macro} , our placer also obtains the macro placement for each benchmark in an acceptable runtime. Besides, compared to

SEU, our algorithm achieves arithmetic mean improvements of 6.0%, 8.3%, and 45.1% in $T_{P\&R}$, S_R , and S_{score} , respectively. Additionally, our algorithm demonstrates geometric mean enhancements of 4.3%, 7.0% and 10.2% in $T_{P\&R}$, S_R and S_{score} , respectively, when compared with SEU. In terms of the final score S_{score} , our algorithm outperforms SEU and Vivado by 45.1% and 62.9% in the arithmetic mean, and by 10.2% and 39.9% in the geometric mean. Besides, Figure 11 provides the detailed comparison of S_R and S_{score} for each benchmark in BC-04. As can be seen from the figure, our algorithm achieves the best routability and final score in most benchmarks, and the smaller extreme values also verify the robustness of our algorithm. Overall, the experimental results demonstrate the effectiveness and robustness of our algorithm for handling complex cascade shape constraints.



(a) S_R



(b) S_{score}

Fig. 11. Comparison of S_{score} and S_R in BC-04.

E. Ablation Studies

In this subsection, ablation experiments are conducted to evaluate the individual and combined contributions of the components in our algorithms.

1) *Impact of two-stage technique*: To verify the effectiveness of our two-stage cascade shape handling technique, we conducted an ablation study comparing the two-stage technique (with pseudo nets) with the single cluster technique (without pseudo nets). As shown in Table VI, D_{avg} represents the average displacement of all macros, and D_{max} represents the maximum displacement. On average, our two-stage technique achieves a 10.7%, 3.3%, and 5.7% decreases in D_{max} , and 3.4%, 4.2%, and 10.4% decreases in D_{avg} by on BC-01, BC-02, and BC-03, respectively. Additionally, S_R improves on

TABLE V
EXPERIMENTAL RESULTS ON BC-04.

Metrics	T_{macro} (min)		$T_{P\&R}$ (hr)		S_{IR}		S_{DR}		S_R		S_{score}	
	Avg.	Geo.	Avg.	Geo.	Avg.	Geo.	Avg.	Geo.	Avg.	Geo.	Avg.	Geo.
Ours	5.44	5.24	0.33	0.33	1.81	1.25	6.40	6.34	11.74	7.99	5.32	2.62
SEU	2.91 -46.5%	2.86 -45.4%	0.35 +6.0%	0.34 +4.3%	1.83 +1.3%	1.32 +5.5%	6.61 +3.3%	6.52 +2.9%	12.72 +8.3%	8.55 +7.0%	7.72 +45.1%	2.88 +10.2%
Vivado [21]	-	-	0.4 +21.4%	0.4 +22.4%	2.06 +13.8%	1.37 +9.1%	6.89 +7.6%	6.71 +5.9%	17.08 +45.5%	9.18 +14.8%	8.67 +62.9%	3.66 +39.9%

average by 2.6%, 4.2%, and 4.7%, S_{score} improves by 4.7%, 5.8%, and 6.8% compared to the single cluster technique. The experimental results demonstrate that the displacement during legalization is significantly lower within our two-stage cascade shape handling technique, leading to optimized placement routability.

TABLE VI
IMPACT OF TWO-STAGE TECHNIQUE ON BC-01, BC-02, AND BC-03.

Benchmarks	Techniques	D_{max}	D_{avg}	S_R	S_{score}
BC-01	Two-stage	19.60	1.48	9.87	3.61
	As single cluster	21.69	1.53	10.13	3.78
	Ratio	+10.7%	+3.4%	+2.6%	+4.7%
BC-02	Two-stage	22.43	1.72	9.89	3.46
	As single cluster	23.17	1.79	10.31	3.66
	Ratio	+3.3%	+4.2%	+4.2%	+5.8%
BC-03	Two-stage	18.02	1.75	10.06	3.70
	As single cluster	19.05	1.93	10.53	3.95
	Ratio	+5.7%	+10.4%	+4.7%	+6.8%

2) *Impact of horizontal pseudo nets*: To evaluate the effectiveness of horizontal pseudo nets to $N \times 2$ cascade shapes, we also conducted an ablation study comparing our algorithm with or without horizontal pseudo-nets on BC-04. As shown in Table VII, our algorithm without horizontal pseudo nets achieves a 4.5% reduction in D_{max} , an 8.8% reduction in D_{avg} , a 3.5% improvement in S_R , and a 6.2% improvement in S_{score} compared to with horizontal pseudo nets. These results demonstrate that avoiding horizontal pseudo nets enhances flexibility in instance spreading, reduces legalization displacement, and ultimately improves the placement routability.

TABLE VII
IMPACT OF HORIZONTAL PSEUDO NETS ON BC-04.

Benchmarks	Techniques	D_{max}	D_{avg}	S_R	S_{score}
BC-04	w pseudo nets	18.86	1.15	11.74	5.32
	w/o pseudo nets	19.72	1.25	12.15	5.65
	Ratio	+4.5%	+8.8%	+3.5%	+6.2%

3) *Impact of ILP-based legalization*: We examined the impact of ILP-based cascade shapes legalization or ILP-based all macros legalization. Table VIII provides a detailed analysis of the runtime and number of macros on BC-01, BC-02, BC-03 and BC-04. This table includes the average runtime for ILP-based cascade shape legalization T_{ILP} , bipartite matching-based macro legalization $T_{Matching}$, and solely using ILP for all

macro legalization $T_{solely\ use\ ILP}$, along with the total number of macros $N_{all\ macros}$, and the ratio of cascade macros to the total number of macros $R_{cascade\ macros}$ across all benchmarks.

On average, T_{ILP} for only cascade shapes takes approximately 1 s. For the remaining macros, $T_{Matching}$ requires significantly less time, averaging around 0.5 s. However, when ILP is applied to all macros, $T_{solely\ use\ ILP}$ can extend to several hundred seconds. This can lead to significant computational overhead, and if the total runtime of macro placement T_{macro} exceeds the 10 minutes time limit, which will lead to runtime penalty in S_{score} defined in Section II-E. Given these challenges, we combine ILP-based region-aware cascade shape legalization and matching-based legalization for the remaining macros.

TABLE VIII
IMPACT OF ILP-BASED MACRO LEGALIZATION.

Benchmarks	T_{ILP}	$T_{matching}$	$T_{solely\ use\ ILP}$	$N_{all\ macros}$	$R_{cascade\ macros}$
BC-01	0.00 s	0.63 s	275 s	2554	0.0 %
BC-02	1.33 s	0.44 s	227 s	2546	4.2 %
BC-03	0.84 s	0.53 s	216 s	2571	14.1 %
BC-04	0.34 s	0.42 s	233 s	2550	2.5 %

V. CONCLUSIONS

In this paper, we have presented an effective and efficient routability-driven macro placement algorithm for modern FPGAs with cascade shapes and region constraints. We first developed a routability-driven mixed-size analytical global placement to evenly distribute both macros and cells while considering cascade shapes and region constraints. Particularly, the proposed global placement engine integrates a well-trained congestion prediction model, targeting benchmarks with high routing congestion to enhance overall routability. Then, we proposed an ILP-based cascade shapes legalization, followed by matching-based macro legalization to remove macro overlaps while satisfying the region constraints. Finally, a routability-driven detailed macro placement was proposed to refine the solution. Compared with the top contestants of the MLCAD 2023 FPGA macro placement contest and state-of-the-art works, experimental results have shown that our algorithm achieves the best overall score and routability. In future work, we plan to explore optimized architecture of our ML-based model to further improve the generalization and efficiency. Additionally, we aim to extend its application to more benchmarks, improving its robustness and applicability.

REFERENCES

- [1] Z. Abuowaimer, D. Maarouf, T. Martin, J. Foxcroft, G. Gréwal, S. Areibi, and A. Vannelli. Gplace3.0: Routability-driven analytic placer for ultrascale fpga architectures. *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, 23(5):1–33, 2018.
- [2] M. B. Alawieh, W. Li, Y. Lin, L. Singhal, M. A. Iyer, and D. Z. Pan. High-definition routing congestion prediction for large-scale fpgas. In *Proceedings of IEEE Asia and South Pacific Design Automation Conference (ASP-DAC)*, pages 26–31, 2020.
- [3] I. Bustany, G. Gasparyan, A. Gupta, A. B. Kahng, M. Kalase, W. Li, and B. Pramanik. The 2023 mlcad fpga macro placement benchmark design suite and contest results. In *Proceedings of ACM/IEEE Workshop on Machine Learning for CAD (MLCAD)*, pages 1–6, 2023.
- [4] Y.-W. Chang, Z.-W. Jiang, and T.-C. Chen. Essential issues in analytical placement algorithms. *IPSI Transactions on System LSI Design Methodology*, pages 145–166, 2009.
- [5] G. Chen, C.-W. Pui, W.-K. Chow, K.-C. Lam, J. Kuang, E. F. Young, and B. Yu. RippleFPGA: Routability-driven simultaneous packing and placement for modern fpgas. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, 37(10):2022–2035, 2018.
- [6] H. Gu, J. Gu, K. Peng, J. Yang, and Z. Ziran. Late breaking results: Routability-driven fpga macro placement considering complex cascade shape and region constraints. In *Proceedings of ACM Design Automation Conference (DAC)*, pages 1–2, 2024.
- [7] J. Gu, Z. Jiang, Y. Lin, and D. Z. Pan. Dreamplace 3.0: Multi-electrostatics based robust vlsi placement with region constraints. In *Proceedings of IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pages 1–9, 2020.
- [8] M.-K. Hsu, Y.-W. Chang, and V. Balabanov. Tsv-aware analytical placement for 3d ic designs. In *Proceedings of ACM/IEEE Design Automation Conference (DAC)*, pages 664–669, 2011.
- [9] W. Li, S. Dhar, and D. Z. Pan. UTPlaceF: A routability-driven fpga placer with physical and congestion aware packing. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, 37(4):869–882, 2018.
- [10] T. Liang, G. Chen, J. Zhao, S. Sinha, and W. Zhang. Amf-placer: High-performance analytical mixed-size placer for fpga. In *Proceedings of IEEE/ACM International Conference On Computer Aided Design (ICCAD)*, pages 1–9, 2021.
- [11] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 10012–10022, 2021.
- [12] J. Lu, P. Chen, C. C. Chang, L. Sha, and C. K. Cheng. ePlace: Electrostatics-based placement using fast fourier transform and nesterov's method. *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, 20(2):1–34, 2015.
- [13] J. Mai, Y. Meng, Z. Di, and Y. Lin. Multi-electrostatic fpga placement considering slicel-slicem heterogeneity and clock feasibility. In *Proceedings of ACM/IEEE Design Automation Conference (DAC)*, pages 649–654, 2022.
- [14] J. Mai, J. Wang, Y. Chen, Z. Guo, X. Jiang, Y. Liang, and Y. Lin. Openparf 3.0: Robust multi-electrostatics based fpga macro placement considering cascaded macros groups and fence regions. In *Proceedings of IEEE International Symposium of Electronics Design Automation (ISED)*, pages 374–379, 2024.
- [15] Y. Meng, W. Li, Y. Lin, and D. Z. Pan. elfplace: Electrostatics-based placement for large-scale heterogeneous fpgas. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, 41(1):155–168, 2022.
- [16] M. Pan, N. Viswanathan, and C. Chu. An efficient and effective detailed placement algorithm. In *Proceedings of IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pages 48–55, 2005.
- [17] J. P. Pedroso. Optimization with gurobi and python. *INESC Porto and Universidade do Porto, Porto, Portugal*, 1, 2011.
- [18] K. Peng and W. Zhu. Pplace-MS: Methodologically faster poisson's equation based mixed-size global placement. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, 2023.
- [19] R. S. Rajarathnam, M. B. Alawieh, Z. Jiang, M. Iyer, and D. Z. Pan. DREAMPlaceFPGA: An open-source analytical placer for large scale heterogeneous fpgas using deep-learning toolkit. In *Proceedings of Asia and South Pacific Design Automation Conference (ASP-DAC)*, pages 300–306, 2022.
- [20] P. Spindler and F. M. Johannes. Fast and accurate routing demand estimation for efficient routability-driven placement. In *Proceedings of ACM/IEEE Design, Automation and Test in Europe Conference (DATE)*, pages 1–6, 2007.
- [21] Xilinx Corporation. Zynq ultrascale+ device technical reference manual. Online, 2022.
- [22] Z. Xiong, R. S. Rajarathnam, Z. Jiang, H. Zhu, and D. Z. Pan. Dreamplacefpga-mp: An open-source gpu-accelerated macro placer for modern fpgas with cascade shapes and region constraints. *arXiv preprint arXiv:2311.08582*, 2023.
- [23] S. Yang, A. Gayasen, C. Mulpuri, S. Reddy, and R. Aggarwal. Routability-driven fpga placement contest. In *Proceedings of ACM International Symposium on Physical Design (ISPD)*, pages 139–143, 2016.
- [24] S. Zheng, L. Zou, P. Xu, S. Liu, B. Yu, and M. Wong. Lay-Net: Grafting netlist knowledge on layout-based congestion prediction. In *Proceedings of IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pages 1–9, 2023.
- [25] Z. Zhu, Y. Mei, Z. Li, J. Lin, J. Chen, J. Yang, and Y.-W. Chang. High-performance placement for large-scale heterogeneous fpgas with clock constraints. In *Proceedings of ACM/IEEE Design Automation Conference (DAC)*, pages 643–648, 2022.



Hao Gu received the M.Sc. degree in Electronic and Communication Engineering from Nanjing University of Posts and Telecommunications, Nanjing, China, in 2021. He is currently pursuing the Ph.D. degree at the School of Integrated Circuits, Southeast University, Nanjing, China.

His research interests focus on machine learning-assisted EDA algorithms, particularly in placement. He received the First Place in the 2023 MLCAD Contest.



Jian Gu received the B.Eng. degree in electronic science and technology from Hohai University, Nanjing, China, in 2021. He completed his master's degree at Southeast University, China.

His research interests include FPGA placement and routing algorithm.



Keyu Peng received the B.Sc. degree in information and computing sciences from Northeast Forestry University, Harbin, China, in 2019, and the M.Sc. degree in operations research and cybernetics from Fuzhou University, Fuzhou, China, in 2023. He is currently pursuing the Ph.D. degree with Southeast University, Nanjing, China.

His research interest is VLSI physical design automation.



Jianli Chen received the B.Sc. degree in information and computing sciences, the M.Sc. degree in computer application technology, and the Ph.D. degree in applied mathematics from Fuzhou University, Fuzhou, China, in 2007, 2009, and 2012, respectively.

He is currently a Professor with the State Key Laboratory of ASIC and System, Fudan University, Shanghai, China. He is the Founder of LEDA Technology, Shanghai. His research interests include optimization theory and applications, and optimization

methods for VLSI physical design automation.

Prof. Chen has received two Best Paper Awards from the International Forum on Operations Research and Control Theory in 2011 and the Design Automation Conference in 2017, the Best Paper Award candidates from DAC three times, and the Best Paper Award candidates from International Conference on Computer-Aided Design (ICCAD) twice. He and his group were the recipient of the First Place Award at the CAD Contest at ICCAD in 2017–2019, respectively. He has also received the Distinguished Young Scholars Foundation of Fujian Educational Committee in 2016, the China Computer Federation Integrated Circuit Early Career Award in 2018, the China Society for Industrial and Applied Mathematics Youth Science Award in 2020, and the First Prize of Shanghai Technology Invention Award in 2020. He has been serving as a Design Automation Technical Committee Member of IEEE Council on Electronic Design Automation since January 2018.



Jun Yang received the B.S. and Ph.D. degrees in electronic engineering from Southeast University, Nanjing, China, in 1999 and 2004, respectively. He is currently a Professor with the National ASIC System Engineering Research Center, Southeast University, Nanjing, China.

He has authored and co-authored over 50 technical articles in conferences and journals, including IEEE International Solid-State Circuits Conference (ISSCC), Design Automation Conference (DAC), the IEEE JOURNAL OF SOLID-STATE CIRCUITS,

IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—I: REGULAR PAPERS (TCASI), IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—II: EXPRESS BRIEFS (TCASII), and IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION (TVLSI) SYSTEMS. He has authorized over 100 Chinese and U.S. invention patents. His current research focuses on static random-access memory (SRAM) design, in-memory computing, and near-threshold design.



Ziran Zhu received the B.Sc. degree in computer science and technology, and the Ph.D. degree in applied mathematics from Fuzhou University, Fuzhou, China, in 2015 and 2020, respectively. He is currently a Lecturer with the National ASIC System Engineering Research Center, Southeast University, Nanjing, China.

His research interests include optimization theory and applications, and VLSI placement and routing. He received the Best Paper Award in DAC 2017, the First Place in CAD Contest @ICCAD 2017 and

2018, the First Place in MLCAD Contest 2023, and the Second Place in CAD Contest @ICCAD 2022.